

Step 5 — Supervisor + Command Routing — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic**, **Intermediate**, **Advanced**.

- [Hint 1 — Shared state, and how Command routing works](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Shared state, and how Command routing works

Basic Version

Every node you've already built (Research, Analysis, Writer, Reviewer) becomes a node in a LangGraph graph. Instead of a fixed arrow saying "this node always goes to that node," each node can return a `Command` object that says two things at once: what to update in the shared state, and which node to go to next. That's the whole idea of `Command`-based routing — the *decision* of where to go next lives inside the node's own return value, decided by a Supervisor node, instead of being drawn as a fixed line on the graph ahead of time.

Think of shared state as one shared clipboard everyone can read and write to, and think carefully about what belongs on that shared clipboard (the topic, the research notes, the draft, the verdict) versus what's just one agent's own scratch thinking that shouldn't leak onto it.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Design `ContentForgeState` (a `TypedDict`) first, before writing any node — every field a specialist needs to read or write goes here: `topic`, `research_notes` (Step 3's `ResearchNotes`), `findings` (Step 4's `Findings`), `draft`, `review_verdict`, `revision_count`, and a `completed_steps` list tracking which specialists have already run. Nothing about *how* a specialist does its job belongs here — only the inputs/outputs each one already had a clear, typed interface for from Steps 1-4.

The standard shape for a `Command`-routed supervisor system: each specialist node does its work, then returns `Command(goto="supervisor", update={...its output fields...})` — handing control back to the Supervisor rather than to the next specialist directly. The Supervisor node reads `state["completed_steps"]` and decides the next hop: `Command(goto="research")`, then later `Command(goto="analysis")`, and so on, until everything is done, then `Command(goto=END)`. This hub-and-spoke shape is exactly why the Supervisor can "keep track of what's already been done" (the README's checklist item) — it's the one node that sees the whole picture, every time.

The one exception: reconnecting Step 2's Writer ↔ Reviewer loop. That loop should stay tight — `Writer → Reviewer → (back to Writer, or on to the Supervisor once done)` — not bounce back through the Supervisor on every single round. The Supervisor only needs to get involved once that whole sub-loop is finished.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Two failure modes only show up once separately-proven pieces get wired together into an actual graph, and neither is hypothetical — they're exactly the first two rows of this project's own problems table.

Redundant work: if the Supervisor's routing logic isn't careful, nothing stops it from sending the same topic to Research twice, or re-running Analysis after Writer has already started drafting from it. `completed_steps` (from Hint 1) is the fix, but only if the Supervisor actually *checks* it before routing, every single time — not just on the first pass. Write the Supervisor's routing as a strict, ordered check against `completed_steps`, not a one-time decision.

A routing bug looping forever: a hard `max_rounds` limit protects the *inner* Writer ↔ Reviewer loop (Step 2 already solved that), but nothing yet protects the *outer* graph if the Supervisor's own logic has a bug — say, a state update that doesn't actually get saved, so `completed_steps` never grows, so the Supervisor keeps routing back to the same node forever. LangGraph's `recursion_limit` (passed in the graph's invoke config, the same mechanism Step 3's Advanced hint used to bound the Research agent's own tool-calling loop) is the outer safety net for exactly this — a graph-wide hop limit that raises a clear error instead of running until you kill the process.

The extra pieces:

- The Supervisor's routing function checks `completed_steps` explicitly for each specialist name before ever routing to it again.
- `graph.invoke(initial_state, {"recursion_limit": N})` — pick `N` generously above your expected hop count (roughly: 1 supervisor hop per specialist, plus every writer/reviewer round, plus a few to spare), so a legitimate run never hits it, but a genuine routing bug does.
- Consider what the *caller* should see if `recursion_limit` is hit — a clear, named exception is far more useful than LangGraph's default `GraphRecursionError` bubbling up unexplained.

Sketch the Supervisor's `completed_steps` check yourself before checking Hint 2.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate design a Supervisor that routes correctly *assuming* everything works as planned. Advanced adds the two things that only matter once you assume something *won't* go as planned — the Supervisor re-running a specialist it didn't need to, and the whole graph looping longer than any real bug should be allowed to run silently.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

```
state.py:
    ContentForgeState: topic, research_notes, findings, draft,
                        review_verdict, revision_count, completed_steps, final_re

agents/supervisor.py:
    supervisor_node(state):
        if "research" not done: go to research
        elif "analysis" not done: go to analysis
        elif "writer_reviewer" not done: go to writer
        else: go to END, save final_report = state's draft

research/analysis nodes:
    do the work (reuse Steps 3/4's functions), save result to state,
    mark this step done, go back to supervisor

writer/reviewer nodes:
    writer: reuse Step 2's run_writer, go to reviewer
    reviewer: reuse Step 2's run_reviewer
             if approved or out of rounds: mark "writer_reviewer" done, go to supervis
             else: go back to writer

graph.py:
    wire all 5 nodes into a StateGraph, entry point = supervisor

main.py:
    run the graph on a real topic, print the final report
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

```
state.py:
class ContentForgeState(TypedDict):
    topic: str
    research_notes: ResearchNotes | None
    findings: Findings | None
    draft: str | None
    review_verdict: ReviewVerdict | None
    revision_count: int
    completed_steps: list[str]
    final_report: str | None

agents/supervisor.py:
def supervisor_node(state: ContentForgeState) -> Command[Literal["research",
done = state["completed_steps"]
if "research" not in done:
    return Command(goto="research")
if "analysis" not in done:
    return Command(goto="analysis")
if "writer_reviewer" not in done:
    return Command(goto="writer")
return Command(goto=END, update={"final_report": state["draft"]})

agents/research_node.py (or inline in graph.py):
def research_node(state) -> Command[Literal["supervisor"]]:
    notes = run_research(state["topic"])
    return Command(
        goto="supervisor",
        update={"research_notes": notes, "completed_steps": state["completed_
    )
# analysis_node follows the same shape, reusing run_analysis

writer/reviewer nodes:
def writer_node(state) -> Command[Literal["reviewer"]]:
    feedback = state["review_verdict"].feedback if state["review_verdict"] el
    draft = run_writer(build_notes_string(state), feedback)
    return Command(goto="reviewer", update={"draft": draft, "revision_count":

def reviewer_node(state) -> Command[Literal["writer", "supervisor"]]:
    verdict = run_reviewer(state["draft"])
    if verdict.approved or state["revision_count"] >= MAX_ROUNDS:
```

```

        return Command(
            goto="supervisor",
            update={"review_verdict": verdict, "completed_steps": state["comp
        )
    return Command(goto="writer", update={"review_verdict": verdict})

graph.py:
    build a StateGraph(ContentForgeState), add all 5 nodes,
    set entry point to "supervisor", compile it

main.py:
    initial_state = {"topic": "...", "research_notes": None, ..., "completed_step
    result = graph.invoke(initial_state)
    print(result["final_report"])

```

Test the full pipeline start to finish on a real topic, plus a case designed to force the Writer ↔ Reviewer loop to hit `MAX_ROUNDS` (reuse Step 2's tricky test case) now running *inside* the full graph. Write it yourself, then compare against the [Solution](#).

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's almost the hardened graph invocation and the Supervisor's redundancy guard — fill in the missing piece yourself:

```
class GraphStuckError(Exception):
    pass

def run_content_forge(topic: str, max_hops: int = 25) -> str:
    initial_state: ContentForgeState = {
        "topic": topic,
        "research_notes": None,
        "findings": None,
        "draft": None,
        "review_verdict": None,
        "revision_count": 0,
        "completed_steps": [],
        "final_report": None,
    }
    try:
        result = graph.invoke(initial_state, {"recursion_limit": max_hops})
    except GraphRecursionError as e:
        raise GraphStuckError(
            f"ContentForge did not finish within {max_hops} hops for topic '{topic}' - "
            "likely a routing bug in the Supervisor, not a slow run."
        ) from e
    return result["final_report"]
```

And in `supervisor.py`, the redundancy guard — make sure this check runs on every call, not just the first:

```
def supervisor_node(state: ContentForgeState):
    done = state["completed_steps"]
    # your turn: for each specialist name in order (research, analysis,
    # writer_reviewer), if it's not yet in `done`, route to it and stop
    # checking further names. This must re-derive the answer from `done`
    # every time this function runs – never cache "what's next" anywhere else.
    ...
```

Fill in the ordered check, then compare all 3 of your finished versions against the [Solution](#).

Difference between Basic, Intermediate, and Advanced: same 5-node, `Command`-routed graph at 3 completeness levels — Basic and Intermediate assume the Supervisor's logic and the graph's execution both behave correctly. Advanced adds the two outer safety nets that only matter when one of them doesn't: a `recursion_limit` that turns a silent infinite loop into a clear, named error, and a redundancy check written so it can never go stale, because it's recomputed from `completed_steps` on every single Supervisor call instead of decided once.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)